

软件过程改进方案设计文档

1. 文档说明

本设计文档依据前期形成的《软件过程定义》和《AI技术应用场景匹配报告》编写，面向“旧的怎么办”校园二手网项目，对现有软件过程进行针对性改进设计。改进目标不是推翻原有过程，而是在保留 Scrum 小团队、6 个 Sprint、双端并行开发、里程碑交付等基本机制的前提下，引入适合课程项目的轻量化 AI 赋能措施，使过程从“能完成项目”进一步提升到“更高效、更稳定、更可控地完成项目”。

本项目原有过程已经覆盖需求、设计、实现、测试、发布与评审等核心活动，整体上具备完整的软件生命周期闭环。但从课程项目和小团队实践角度看，仍存在几个典型短板：需求阶段对文字描述依赖较强，容易产生歧义；设计阶段产物形成较慢，图和文档容易与实现脱节；实现阶段跨前后端和双端联调，人工沟通成本高；测试阶段主要依赖集中回归，边界场景覆盖不足；发布和演示前的问题定位偏依赖人工经验。因此，本次过程改进的重点是利用 AI 技术增强这些高重复、高风险、易遗漏的活动环节。

2. 改进背景与问题分析

结合项目计划、软件过程定义以及 AI 技术应用场景匹配结果，可以将当前软件过程中的改进需求概括为以下五类问题。

第一，需求活动的结构化程度不足。当前项目在 M0 阶段由项目经理牵头完成 PRD、原型和 UI 设计稿，但原始需求往往来自讨论、经验判断和功能设想，需求表达容易出现不统一、不完整或边界场景遗漏。例如“商品下架”“订单关闭”“评价完成”等状态在不同成员理解中可能不完全一致，后续容易引发设计返工和测试争议。

第二，设计活动的产物生成效率偏低。虽然项目已经定义了架构与详细设计过程，但小团队通常倾向于“先做出功能再补文档”，导致接口文档、类关系、时序流程、数据库结构说明等设计产物更新滞后。这样会直接影响前后端协作效率，也会降低成员之间的共享理解。

第三，实现与集成活动存在明显的人力瓶颈。项目在 Sprint 2 到 Sprint 4 期间同时推进后端、PC 端和微信小程序端，工作量大、节奏快，联调频繁。如果仍然主要依赖人工编码、人工对齐接口、人工代码评审，就容易出现提交速度快但质量波动大、后期修复成本高的问题。

第四，测试与验证活动偏后置。当前过程规定 Sprint 4 做回归测试、Sprint 5 做全量系统测试，这种安排本身没有问题，但如果早期缺少测试点自动生成、风险模块识别和边界场景补足，就会造成测试阶段压力集中，甚至在答辩前才发现关键缺陷。

第五，发布与交付阶段排障效率有待提升。课程项目虽然规模不大，但一旦进入测试环境、演示环境或生产环境，问题往往表现为跨层联动，例如“前端报错 + 后端超时 + Redis 未命中 + 权限配置异常”共同出现。此时纯靠人工看日志和经验定位，效率较低，容易耽误演示节奏。

因此，软件过程改进的核心任务是：在不增加大量额外管理负担的前提下，把 AI 技术嵌入现有流程的关键节点，实现“前移质量保障、减少重复劳动、提升联调效率、增强测试覆盖、提高交付稳定性”。

3. 改进目标

本次软件过程改进面向课程项目实际，设定以下五项目标：

1. 提高需求活动质量，使需求描述、验收标准和状态定义更一致，减少后续返工。
2. 提升设计活动效率，使需求、架构、接口、数据模型之间的转换更快、更规范。

3. 提升开发阶段产出效率，在保证代码质量的前提下缩短实现和联调周期。
4. 提高测试阶段覆盖率与问题发现能力，把部分质量问题提前到编码后和集成前发现。
5. 提升发布与演示阶段的稳定性，使故障定位、异常分析和性能优化更高效。

进一步说，本次改进追求的不是“完全自动化开发”，而是构建一个“人主导、AI 辅助、关键节点设门禁”的增强型软件过程。也就是说，团队成员仍对需求、设计、实现和发布结果负责，但 AI 在信息整理、产物生成、问题提示、风险识别和分析判断方面承担强辅助角色。

4. 改进原则

为保证改进方案适合重点本科课程项目，本方案遵循以下原则。

4.1 轻量化原则

不引入重型组织级制度，不额外增加大量审批和文档。AI 赋能应尽量嵌入现有 Scrum 节奏中，例如放进 Sprint 计划会、需求整理、开发提交前检查、回归测试前准备、答辩前巡检等活动，而不是重新发明一套复杂流程。

4.2 高收益优先原则

优先改进工作量大、错误代价高、重复劳动多的环节。根据前期报告，优先级最高的环节是实现与集成、测试与验证，其次是需求分析和设计，再其次是发布与交付阶段的异常分析。

4.3 人机协同原则

AI 负责生成、建议、检查与分析，团队成员负责确认、取舍和最终决策。需求是否采纳、架构是否成立、代码是否合并、缺陷是否关闭，仍由人来做最终判断，避免因过度依赖 AI 导致错误放大。

4.4 可度量原则

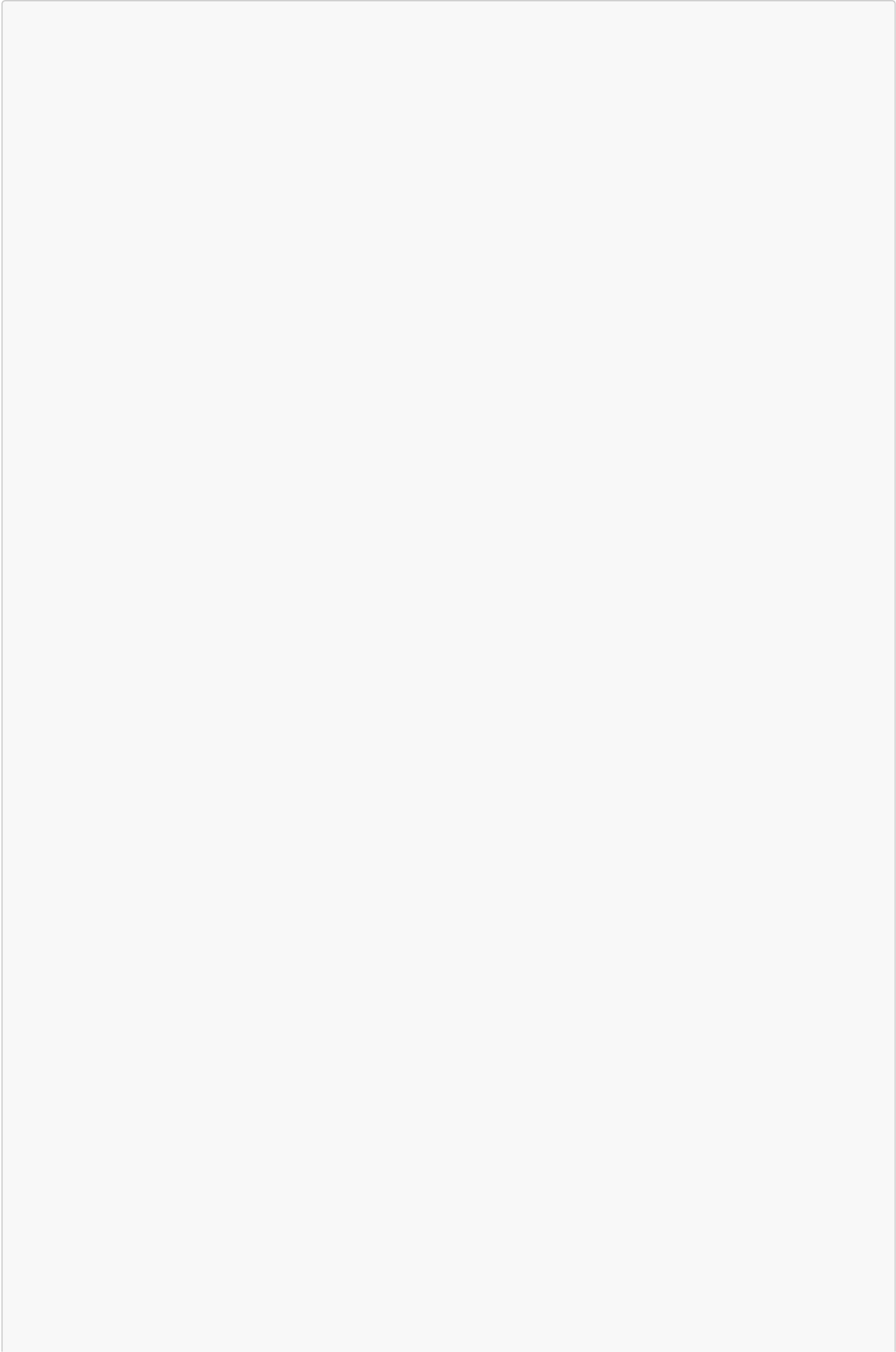
改进不能只写成概念口号，而要通过明确指标衡量成效，例如 PRD 编写时间缩短比例、设计产物补齐速度、代码评审缺陷前移率、测试用例覆盖增长、缺陷定位耗时下降等。

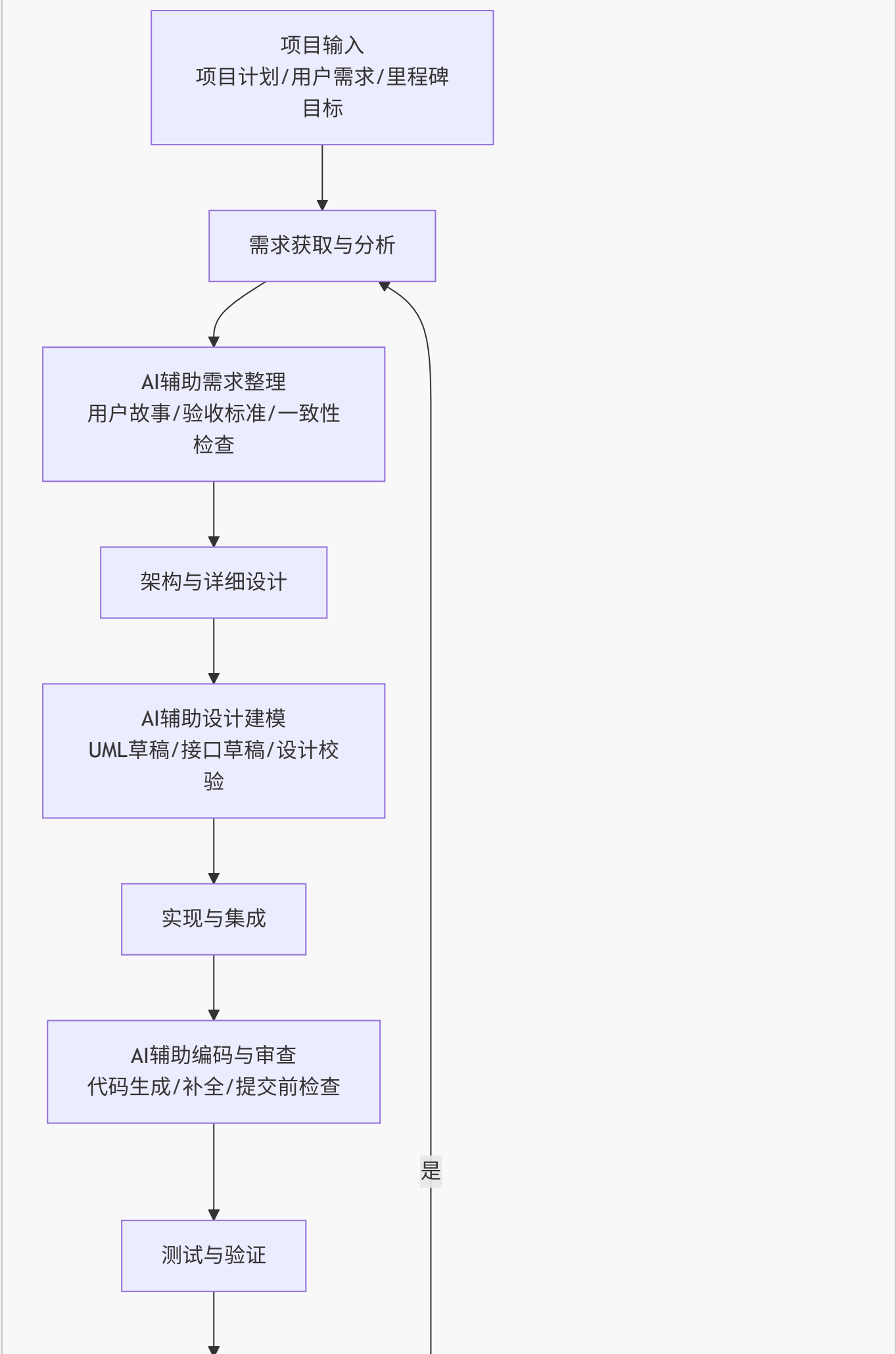
5. 改进后的软件过程总体模型

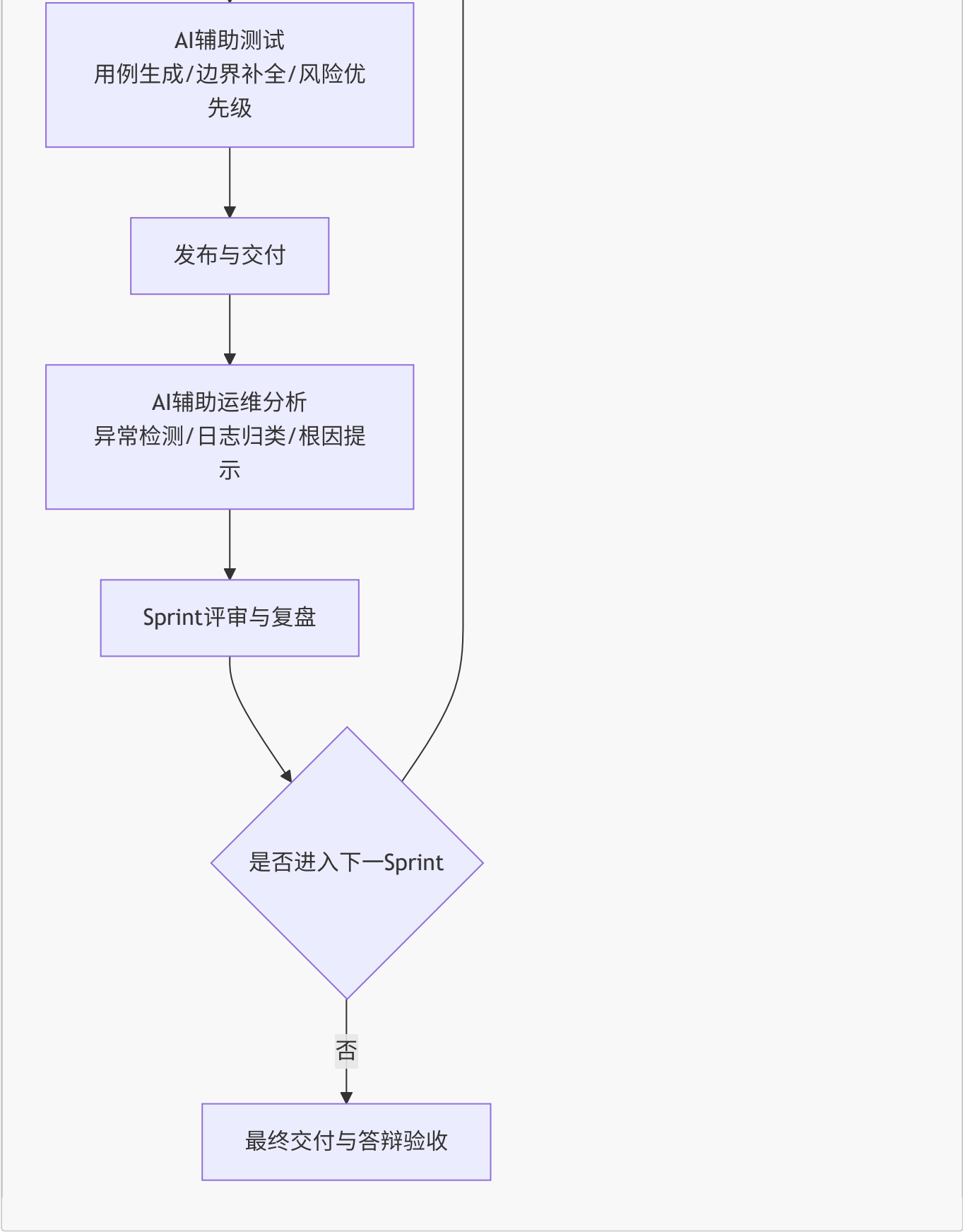
在保留原有“需求获取与分析 -> 架构与详细设计 -> 实现与集成 -> 测试与验证 -> 发布与交付 -> Sprint 评审复盘”主线的基础上，本方案新增五个 AI 赋能控制点：

1. 需求整理与一致性校验点
2. 设计建模与架构校验点
3. 编码生成与提交前智能审查点
4. 测试用例生成与风险优先测试点
5. 发布前异常检测与根因辅助分析点

改进后的总体流程如下：







该流程的含义是：原有生命周期过程不变，但在每个阶段增加一个“AI 增强层”，用于提高阶段内活动质量，并把部分问题前移发现。这样既能保持团队熟悉的开发方式，又能让 AI 真正发挥作用。

6. 各过程的改进设计

6.1 需求获取与分析过程改进

原过程重点是输出 PRD、原型和 UI 设计稿，改进后的做法是增加“AI 需求整理与一致性校验”机制。

具体做法包括：

- 将需求访谈记录、群聊讨论、飞书纪要、产品想法输入 AI，自动整理为用户故事、功能点清单和验收标准草稿。
- 使用 AI 检查需求中的歧义词、状态冲突、角色职责混淆和异常流程遗漏。
- 对“商品、订单、评价、审核、在线沟通”等核心对象建立术语表，确保各成员描述一致。
- 在 Sprint 计划会前，由项目经理和产品负责人用 AI 生成迭代需求摘要，减少会议整理成本。

改进效果体现在：需求文档形成速度更快，需求描述更规范，设计输入更稳定。对于校园二手网项目，这意味着商品发布、订单状态、审核规则等关键业务更不容易在开发后期返工。

6.2 架构与详细设计过程改进

原过程要求明确技术架构、数据库结构和接口规范，改进后的重点是引入“AI 设计建模与架构校验”。

具体做法包括：

- 根据需求说明自动生成用例图、类图、时序图、接口文档草稿，作为人工修订基础。
- 对 Spring Boot + Vue + 小程序 + MySQL + Redis 架构进行职责分层检查，提示可能存在的耦合问题。
- 对数据库实体与接口字段进行一致性检查，减少“表结构改了但接口文档未同步”的情况。
- 对缓存使用、订单状态流转、权限控制等关键设计点进行风险提问和方案补充。

这样改进后，设计阶段不再只是“补文档”，而是形成一个“快速建模 + 快速审视 + 快速对齐”的循环。对课程项目来说，这种做法比重型设计评审更适合，因为它既控制了成本，又提高了设计产物的可用性。

6.3 实现与集成过程改进

这是本项目最重要的改进环节，也是预期收益最大的环节。

原过程主要依赖人工编码和人工联调，改进后引入“AI 编码生成 + 提交前智能审查 + 联调前规范检查”三层机制。

具体做法包括：

- 对标准化程度高的模块使用 AI 辅助生成初始代码，如 CRUD 接口、实体类、DTO、VO、前端表单、接口请求封装等。
- 在编码阶段使用 AI 自动补全、重构建议、重复逻辑识别，提高实现速度。
- 在每次提交前由 AI 对改动进行扫描，检查异常处理、空指针风险、SQL 注入风险、权限遗漏、命名不统一、接口字段不匹配等问题。
- 在前后端联调前由 AI 检查接口定义、状态码约定、参数命名和返回结构是否一致。

这一过程改进的关键意义在于把很多原本会在联调或测试阶段才暴露的问题前移到编码阶段。这样一来，团队不仅写代码更快，而且“改 Bug 的时间”也会减少，整体 Sprint 节奏更稳定。

6.4 测试与验证过程改进

原过程主要在 Sprint 4 和 Sprint 5 做集中测试，改进后增加“AI 测试点自动生成 + 风险优先级测试 + 结果辅助分析”。

具体做法包括：

- 根据 PRD、接口文档和代码变更自动生成功能测试点、异常测试点和边界值测试点。
- 对核心路径，如登录、商品发布、商品详情、下单、评价、后台审核等，自动补充测试场景。
- 根据最近改动范围和历史缺陷分布识别高风险模块，优先安排测试资源。
- 对回归测试失败记录、接口错误信息和日志输出做自动归类，辅助测试人员判断问题来源。

这一改进方案特别适合小团队。因为学生项目最稀缺的不是“测试意识”，而是“测试时间”。AI 的作用不是取代测试，而是帮助团队更快生成测试基础材料，把有限时间放在更关键、更复杂的验证任务上。

6.5 发布与交付过程改进

原过程的重点是部署、验收、答辩材料准备，改进后新增“发布前 AI 巡检与异常分析”。

具体做法包括：

- 在发布前对日志、接口响应、缓存命中情况和数据库连接情况做异常模式识别。
- 对测试环境或预发布环境中的报错信息进行智能归类，提示可能根因。
- 对性能波动进行初步分析，帮助团队识别接口瓶颈、缓存失效和重复请求问题。
- 在中期答辩和最终答辩前进行一次“AI 辅助演示巡检”，重点检查登录态、核心交易流、审核流和演示脚本中的薄弱环节。

这部分改进的价值不在于建立完整的企业级 AIOps 平台，而在于让项目在关键展示节点更稳定、更容易排障。

7. 改进后角色分工设计

为了让过程改进真正可执行，还需要对角色分工做出适配。

- 项目经理 / 产品负责人：负责组织需求整理、确认 AI 输出的用户故事与验收标准、统筹迭代节奏。
- 后端负责人：负责审查 AI 生成的接口、业务逻辑和数据库访问代码，确保符合后端架构约束。
- 数据与中间件负责人 / 运维负责人：负责审查缓存策略、数据库设计、部署日志与发布阶段异常分析结果。
- PC 前端负责人：负责审查 AI 生成的页面结构、组件逻辑和交互流程，保证双端风格一致。
- 全员共同职责：不直接照搬 AI 输出，必须经过人工确认后再纳入正式产物。

换句话说，AI 的引入并不改变原有团队角色，而是改变每个角色完成任务的方式，使其从“纯手工生产者”逐步转向“人机协同的质量控制者”。

8. 实施路线设计

考虑到课程项目周期有限，改进不宜一次全部铺开，建议采用分 Sprint 渐进式落地。

8.1 Sprint 1：先落地需求与编码辅助

优先引入 AI 需求整理、一致性检查、基础代码生成和提交前审查。因为这两个环节最容易快速见效，也最能缓解团队初期压力。

8.2 Sprint 2：补充设计建模与接口一致性检查

在需求已基本稳定、核心架构开始成型后，引入 AI 生成 UML 草稿、接口说明草稿和设计校验功能，帮助团队减少沟通成本。

8.3 Sprint 3：强化测试生成与风险优先测试

当功能模块逐渐增多后，使用 AI 自动生成测试点、补充边界场景，并根据代码改动优先测试高风险区域。

8.4 Sprint 4 及以后：加入发布前巡检与异常分析

在中期答辩版本、正式版和最终验收版本前，重点使用 AI 进行日志分析、问题归类和演示前巡检，提升交付稳定性。

这种实施路线的优点是投入小、收益快、风险低，符合课程项目边做边交付的节奏。

9. 度量指标与预期效果

为验证改进方案是否有效，建议从以下几个方面进行度量：

- 需求阶段：PRD 初稿编写时间、需求变更次数、需求争议条目数。
- 设计阶段：接口文档补齐时间、设计文档与实现不一致次数、联调前接口问题数。
- 开发阶段：单个功能开发耗时、提交前发现的问题数、代码评审平均耗时。
- 测试阶段：测试用例数量、边界场景覆盖数、关键缺陷发现提前率、回归测试通过率。
- 发布阶段：问题定位平均耗时、演示前临时故障次数、版本稳定性评价。

结合前期调研结果，改进后的预期效果可以描述为：

1. 需求整理与 PRD 输出效率明显提升，需求返工率下降。
2. 设计产物形成速度加快，前后端协同成本下降。
3. 代码生成和审查效率提升，质量问题更早暴露。
4. 测试覆盖率提升，关键缺陷在答辩前的暴露率提高。
5. 发布与演示阶段排障时间缩短，系统稳定性增强。

10. 风险与控制措施

任何过程改进都伴随风险，本方案主要面临以下问题。

第一，AI 输出可能存在幻觉或不符合项目上下文。如果成员不加辨别直接采用，反而会引入错误。控制措施是明确“AI 只提供候选结果，正式产物必须人工确认”。

第二，团队成员可能对 AI 过度依赖，削弱自主分析能力。控制措施是要求对关键设计决策、关键业务逻辑和测试结论保留人工解释与复核记录。

第三，工具切换成本可能造成短期效率下降。控制措施是先从需求整理、代码生成、测试点生成等低门槛场景切入，再逐步扩大。

第四，课程项目时间有限，不能把主要精力花在折腾工具上。控制措施是坚持“服务交付优先”，凡是不能直接提升项目产出的 AI 活动，一律不做重投入。

11. 结论

本次《软件过程改进方案设计文档》并不是在原有软件过程之外另起炉灶，而是在“旧的怎么办”校园二手网原有 Scrum 迭代过程之上，设计一套适合 4 人课程项目团队的 AI 增强型软件过程。该过程以需求、设计、实现、测试、发布五大关键阶段为主线，在每个阶段嵌入一个高收益 AI 赋能点，形成“原有过程主线 + AI 辅助增强层 + 人工确认门禁”的改进模型。

从整体看，该方案既保留了原过程的可执行性与简洁性，又吸收了 2024-2026 年 AI4SE 的最新趋势，尤其突出需求一致性校验、设计建模、代码生成与智能审查、测试用例自动生成、高风险缺陷预测和发布前异常分析等能力。对于本项目而言，这样的改进方案最现实、最可落地，也最有可能在有限的课程周期内体现出“效率提升、质量增强、答辩更稳”的实际效果。